IMSAXES

A PROJECT REPORT

Submitted by

ARPILKUMAR SANJAYBHAI KHUNT

220170107055

In partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER ENGINEERING

VISHWAKARMA GOVERNMENT ENGINEERING COLLEGE CHANDKHEDA, AHMEDABAD

Gujarat Technological University, Ahmedabad

April, 2026

Vishwakarma Government Engineering College

Near Visit circle, Sabarmati-Koba highway, Chandkheda, Ahmedabad, Gujarat, India-382424

CERTIFICATE

This is to certify that the project report submitted along with the project entitled **IMSAXES** has been carried out by **Khunt Arpilkumar Sanjaybhai (220170107055)** under my guidance in partial fulfilment for the degree of Bachelor of Engineering in Computer Engineering, 8th Semester of Gujarat Technological University, Ahmadabad during the academic year 2025-26.

Prof. Jaimin M. Shroff
Internal Guide

Prof. Kajal S. Patel
Head of the Department

Vishwakarma Government Engineering College Chandkheda, Ahmedabad

DECLARATION

I hereby declare that the Internship/ Project Report submitted along with the Internship entitled **IMSAXES** submitted in partial fulfillment for the degree of Bachelor of Engineering in Computer Engineering to Gujarat Technological University, Ahmadabad, is a Bonafide record of original project work carried out by me at **Meditab** under the supervision of **Nevil Patel** and that no part of this report has been directly copied from any student's reports or taken from any other source, without providing due reference.

Name of Student — Sign Student

ARPILKUMAR SANJAYBHAI KHUNT _____

ACKNOWLEDGEMENT

I would like to take this opportunity to extend my heartfelt appreciation to everyone who played a role in the successful completion of my internship project.

First and foremost, I am deeply grateful to my internal mentor, **Prof. Jaimin M. Shroff**, for her constant encouragement, thoughtful feedback, and valuable insights throughout this journey. Her unwavering support, guidance, and deep knowledge have been instrumental in shaping the outcome of this project.

I would also like to thank my external guide, **Nevil Patel** at **Meditab**, for his professional guidance and ongoing support during my time at the organisation. His practical suggestions and mentorship greatly enriched my learning experience.

My sincere thanks go to **Prof. Kajal S. Patel**, Head of the Department, for her encouraging words, constructive feedback, and continued motivation throughout the internship.

Lastly, I would like to acknowledge the efforts of all my faculty members and express my gratitude to my friends and peers for their cooperation and assistance, which contributed significantly to the successful completion of this project.

Thank you,

ARPILKUMAR SANJAYBHAI KHUNT

220170107055

ABSTRACT

Purpose of Internship:

The internship provided a valuable opportunity to develop real-world technical and soft skills through hands-on project work. It helped in strengthening problem-solving capabilities, communication, and collaborative skills while gaining deep insights into test automation frameworks and practices.

Scope of Internship:

*The scope included developing robust and scalable automation frameworks using tools such as **Selenium**, **TestNG**, **Maven**, **Extent Reports**, and **UFT**. The focus was on creating maintainable and reusable scripts capable of handling a wide range of testing scenarios, including both sanity and regression testing.*

Work Done:

The internship involved two primary projects:

1. IMSAXES:
   Worked on the development of automation scripts for a company-internal platform named "AXES" (Automation Execution Environment for Selenium and IMS). The system automated sanity and IMS testing of internal web applications using test suites provided by the QA team. The scripts were modular and designed to ensure minimal human intervention during execution.
2. Smart Support System (Quickcap Support Bot):
   Smart Support System (Quickcap Support Bot) is an AI-powered application built using a Retrieval-Augmented Generation (RAG) approach to deliver accurate and context-aware responses. The system consists of two main modules: an ingestion
   pipeline that processes PDFs and other resources to build a dynamic knowledge base, and a conversational interface for real-time user interaction. It leverages streaming capabilities via Streamlit to provide responses word-by-word, enhancing user experience with minimal latency.

Together, both projects contributed to a significant reduction in manual testing efforts and knowledge retrieval, increased test coverage, and enhanced the maintainability of test suites - thereby improving overall product quality and testing efficiency.

List of Figures

List of Tables

Abbreviations

Abbreviation	**Full Form**
API	Application Programming Interface
SQL	Structured Query Language
IMS	Intelligent Medical Software
AXES	Automation Export Environmental Software
EHR	Electronic Health Record
IDE	Intelligent Development Environment
JDK	Java Development Kit
QA	Quality Assurance

Table of Contents

Chapter 1: Overview of the Company

This chapter will introduce the company where I completed my internship. Following the company overview, I will describe the training and learning experiences during the internship period. Each chapter in this report covers a dedicated topic that I explored during my training, supported by real-world examples and practical implementations. At the end of each chapter, I have highlighted the key outcomes for quick recall. This chapter marks the beginning of the journey by providing an introduction to Meditab Software Pvt. Ltd.

Figure 1.1 Company Logo

1.1 About the Company

Meditab Software Pvt. Ltd. is a private software solutions company specialising in healthcare technology. Founded in **1998**, Meditab's mission is to provide advanced, intuitive solutions that help healthcare providers improve patient care and streamline their practice operations. The company is well known for its flagship product, **Intelligent Medical Software (IMS)**, a comprehensive multispecialty EHR (Electronic Health Record) and practice management system.

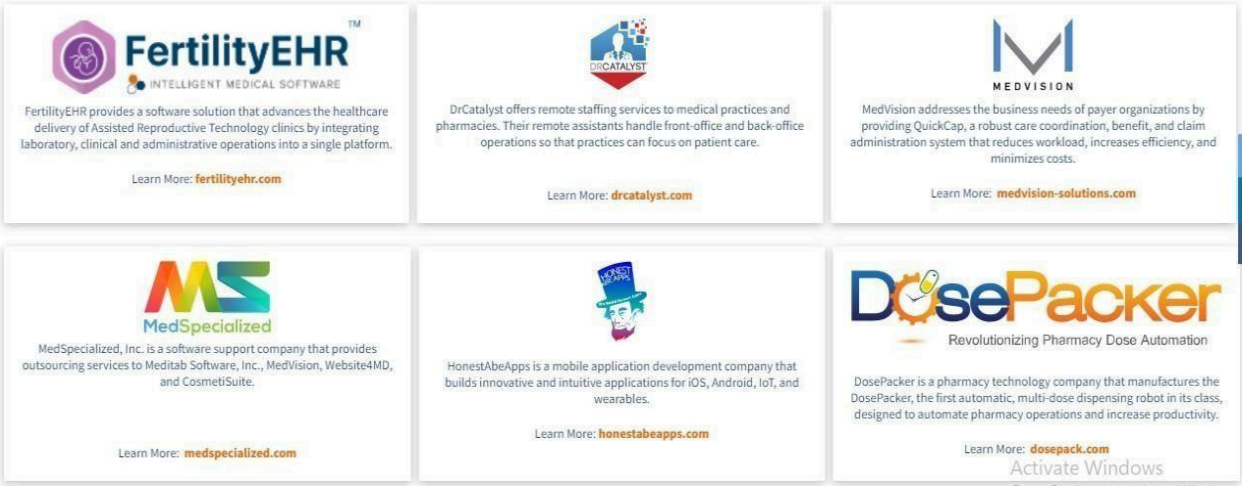


Figure 1.2 Company Product

1.2 Key Offerings

- IMS Platform

  Includes practice management, billing, patient engagement, and telemedicine features.

- Specialty EHRs

  Customised for over 40 medical specialities, including:
  - Allergy & Immunology
  - Fertility
  - Ophthalmology
  - ENT, Dermatology, and more

- AXES Automation Platform

  A Selenium n-based internal tool used for automated testing, script execution, and UI tracking.

- Customisation & Cloud Access

  Highly adaptable software with mobile and cloud capabilities.

- Patient Education Tools

  Provides patients with condition-specific educational content to support their care journey.

1.3 Company Growth and Recognition

- Global Offices in:
  - United States (Headquartered in Sacramento, CA)
  - India
  - Philippines
  - Argentina

- Recognitions
  - ISO 9001:2015 certified
  - Listed in Black Book Rankings
  - Winner of the 2021 Best of Sacramento Award

1.4 Scope of Work

Meditab delivers a wide range of healthcare software solutions, including:

- Multispecialty EHR systems
- Billing and practice management tools
- Telemedicine and patient engagement apps
- Automation testing frameworks (e.g., AXES)

Organization Chart

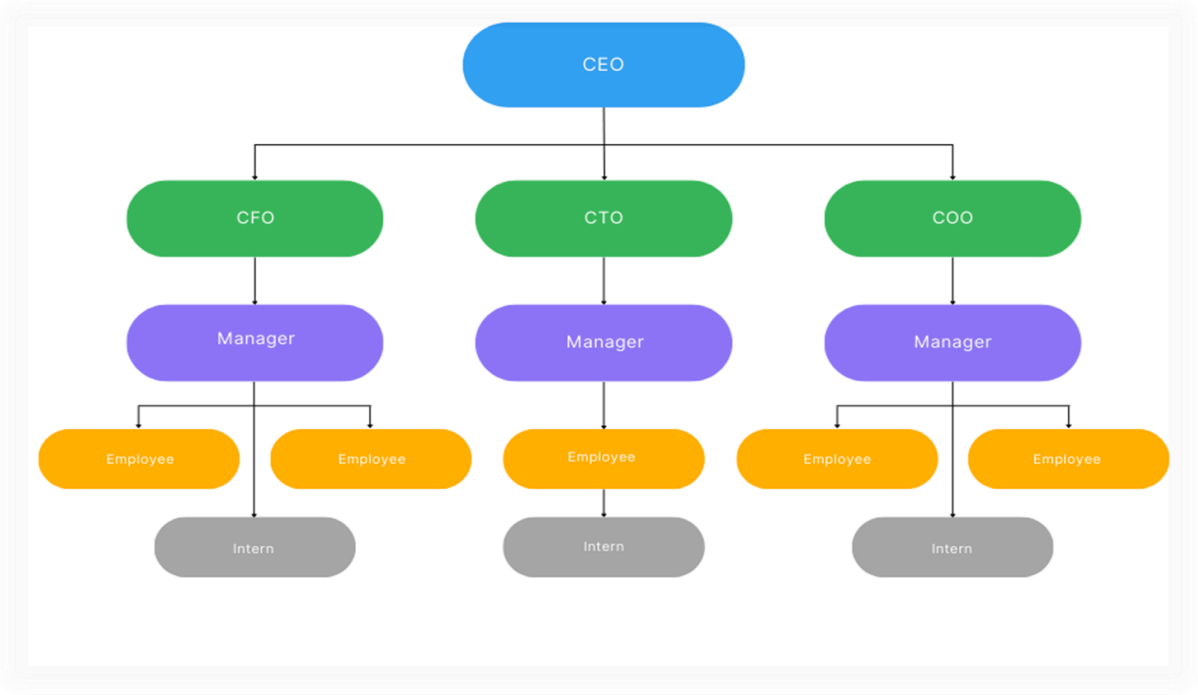


Figure 1.3 Organization Chart

1.5 Capacity Of Plant

Currently our company holds over 2600 employees. But as the company is growing rapidly its capacity is going higher and higher.

Chapter 2: Overview of Internship

2.1. Internship Summary and Project Detail

During my internship at Meditab Software Pvt. Ltd., I was provided with the opportunity to work on real-time software automation under the guidance of experienced professionals. The primary focus of the internship was the development of a dynamic automation testing framework named **IMS AXES**. This framework was aimed at streamlining and enhancing the internal QA processes by automating major workflows of Meditab's web applications.

The internship was thoughtfully structured, offering me well-rounded technical exposure across different domains:

- **SQL Training:** I learned how to work with structured databases, write complex queries, perform joins, filtering, and aggregate operations using SQL. This helped in understanding backend data validation during test automation.

- **Java Training:** I was introduced to object-oriented programming concepts in Java, learning class structures, exception handling, file handling, and API integration. Java formed the backbone of our test scripts in the AXES framework.

- **Selenium & Automation Training:** In-depth training in Selenium WebDriver enabled me to write scripts for browser-based automation. I also explored concepts like dynamic element handling, waits, cross-browser compatibility, and framework integration with TestNG and Maven.

- **UFT Framework Training :** The UFT (Unified Functional Testing) Framework Training provided hands-on experience with this enterprise-level automation tool, focusing on leveraging its object repository and checkpoint capabilities. This training was essential for understanding how to integrate
  diverse testing tools to create a comprehensive and robust hybrid automation framework.

- **DevOps Training :** DevOps Training provided foundational knowledge in continuous integration and continuous deployment (CI/CD) practices. This included hands-on experience setting up a complete CI/CD pipeline using **GitLab Runner**. A key focus was on containerization, utilizing **Docker** to package applications, and orchestration using **Kubernetes** to manage and deploy these containers at scale in an automated manner.

- **Langchain & LangGraph Training :** This training focused on leveraging large language models (LLMs) to build sophisticated, data-aware applications. We learned how to use **Langchain** for chaining multiple
  components (like prompts, LLMs, and external data sources) and **LangGraph** for defining complex, stateful LLM workflows and agentic systems, enabling more reliable and controlled interactions.

This combined training enabled me to understand the full software testing lifecycle and contribute actively to the AXES Automation project.

2.2 Project Overview

The IMS AXES project focuses on the development of a dynamic and reusable test automation framework for internal web applications and window applications used at Meditab. The goal is to reduce repetitive manual testing tasks by automating key application workflows such as login, task management, and verification processes. This system ensures that each deployment or update of the application can be validated quickly with minimal effort using automated scripts. As a result, it enhances the overall product quality and helps provide a seamless user experience to both internal stakeholders and clients.

2.3 Purpose of Project

The primary purpose of this project is to improve testing efficiency and depth through reusable, robust automation scripts. It aims to reduce time spent on repetitive testing cycles and minimise human error. By implementing a reliable automation suite, the project ensures consistency across test runs and builds a professional image in front of the client by delivering a smooth and error-free experience.

2.4 Scope of Project

This automation framework can be applied to test any web-based or desktop-based system using a dynamic, centrally-managed scripting approach. IMS AXES is scalable, allowing multiple test cases and modules to be handled by a single script logic that adapts to different UI flows.

The framework supports:

- **IMS Testing** - Basic validation to ensure stable builds
- **Regression Testing** - Comprehensive testing after updates
- **Cross-module Reusability** - One script across similar workflows
- **Batch Execution** - Automating multiple test runs together

AXES is adaptable and client-agnostic, making it viable for internal and external systems.

2.5 Objective

Two major causes of software failures are poor user experience and inadequate testing. Applications that are not rigorously tested can suffer performance issues, bugs, or even data breaches. IMS AXES aims to address these problems by introducing a highly modular and scalable automation suite.

Key objectives include:

- Improve test coverage with minimal human intervention
- Provide quick feedback loops during CI/CD deployments
- Ensure application reliability under varied scenarios
- Reuse test scripts across different clients and modules

The project also focuses on reproducibility and flexibility. Once the framework is in place, it can be updated with minimal changes to support new test cases.

2.6 Technology And Tools

- **Programming Language:** Java, JavaScript
- **Tools & Frameworks:**
    - **Eclipse IDE:** For writing and managing the codebase
    - **UFT:** For windows applications automation
    - **Maven:** For dependency management and project structure
    - **TestNG:** For designing test suites and generating reports
    - **Extent Reports:** For detailed execution reporting with pass/fail logs
    - **Web Development:** React.js, Express.js, Node.js
    - **DataBase:** PostgreSQL

These tools are chosen for their reliability, strong community support, and scalability in enterprise-level automation environments.

IMS AXES design supports modularity, dynamic object handling (e.g., dropdowns, calendars), and easy integration into existing DevOps pipelines.

Chapter 3: Project Structure and Management

3.1 Project Planning

3.1.1 Project Development Approach

The IMS AXES project followed a structured, iterative, and incremental development approach. It was divided into the following key phases to ensure effective delivery, quality assurance, and long-term maintainability:

Phase 1 – Initiation and Strategy Setup

- Define testing strategy and scope
- Gather automation feasibility and system requirements
- Identify manual components to convert to automation
- Select automation tools: **Selenium**, **UFT**, **Jenkins**, etc.

Phase 2 – Requirements Analysis and Test Planning

- Define testing requirements:
  - Unit Testing
  - Functional Testing
  - Interface Testing
  - Security and Compliance
  - Platform Compatibility
  - Performance Benchmarks
- Understand globalization and accessibility needs
- Estimate development efforts and pricing
- Obtain planning approvals and scheduling sign-offs
- Establish communication between development and QA teams

Phase 3 – Knowledge Transfer and Scenario Design

- Application demonstration and feature walkthrough
- Domain-specific knowledge transfer
- Study user/operations manuals
- Draft test plans and define test cases for key scenarios

Phase 4 – Environment and Platform Setup

- Provisioning of required hardware/software resources
- Setup and configuration of test environments
- Define execution platforms for global testing
- Design and build test suites

Phase 5 – Script Design and Automation Workflow

- Develop test architecture and script framework
- Identify reusable components (object repository, test data)
- Build test cases aligned with the designed workflows
- Execute performance scripts for worst-case scenarios
- Perform repeated test runs with varying data

Phase 6 – Execution and Final Delivery

- Execute full test suites and validate results
- Log all test outcomes with pass/fail criteria
- Track issues and update automation scripts

Phase 7 – Maintenance and Continuous Improvement

- Improve test coverage and reduce script execution time
- Add new test cases based on feature updates
- Maintain modularity and reusability for future automation efforts

3.1.2 Project Efforts and Cost Estimation

To estimate the cost and effort required for this project, we applied the **COCOMO (Constructive Cost Estimation Model)**. This model helps calculate project effort based on the complexity and size of the software in terms of **Function Points (FP)** and **Lines of Code (LOC)**.

COCOMO Classification:

The AXES Automation framework falls under the **"Application"** project category in the COCOMO model due to its purpose of enhancing QA through automation tools.

Function Point Analysis:

Parameter	Count	Simple	Average	Complex	FP Value
No. of User Inputs	22	3	4	6	88.5
No. of User Outputs	5	4	5	7	25.0
No. of Inquiries	3	3	4	6	12.0
No. of Files	8	7	10	15	80.0
External Interfaces	2	5	7	10	14.0

Table 3.1 – Function Point Calculation for COCOMO Model

Question	0	1	2	3	4	5
1. Does the system require reliable backup and recovery?	○	○	○	○	○	◉
2. Are data communications required?	○	○	○	○	◉	○
3. Are there distributed processing functions?	○	○	◉	○	○	○
4. Is performance critical?	○	○	○	○	○	◉
5. Will the system run in an existing, heavily utilized operational environment?	○	○	○	○	○	◉
6. Does the system require on-line data entry?	○	○	○	○	◉	○
7. Does the on-line data entry require the input transaction to be built over multiple screens or operations?	○	○	○	◉	○	○
8. Are the master file updated on-line?	○	○	○	○	◉	○
9. Are the inputs, outputs, files, or inquiries complex?	○	○	○	○	◉	○
10. Is the internal processing complex?	○	○	○	◉	○	○
11. In the code designed to be reusable?	○	○	○	○	○	◉
12. Are conversion and installation included in the design?	○	○	○	○	◉	○
13. Is the system designed for multiple installations in different organizations?	○	○	○	○	○	◉
14. Is the application designed to facilitate change and ease of use by the user?	○	○	○	○	○	◉
Total 58.00						

Fig 3.1 – Questions Dependent on the Project

The Function Points is: Show Function Points 269.37

Fig 3.2 – Function Points Calculation

Programming Language	LOC/FP (average)	Select
Assembly Language	320	○
C	128	○
COBOL	105	○
Fortran	105	○
Pascal	90	○
Ada	70	○
Object-Oriented Languages	30	◉
Fourth Generation Languages (4GLs)	20	○
Code Generators	15	○
Spreadsheets	6	○
Graphical Languages (icons)	4	○

LOC/F P: Show LOC/FP 8081.10

Software Project	a_b	b_b	c_b	d_b	Select
Organic	2.4	1.05	2.5	0.38	○
Semi-detached	3.0	1.12	2.5	0.35	○
Embedded	3.6	1.20	2.5	0.32	◉

Calculate Effort and Duration

Effort (E) = $a_b(KLOC)^{b}_{b}$ = 44.18 **Duration (D) = $c_b(E)^{d}_{b}$ =** 8.40

Fig 3.3 – LOC/FP Calculation Diagram

3.1.3 Roles and Responsibilities

Name	*Role*
Arpil Sanjaybhai Khunt	Requirement Gathering, Script Design, Test Execution, Reporting, Designing, Documentation

3.2 Project Scheduling and Representation

Project scheduling was executed using a **Gantt Chart**, a type of bar chart used to represent the timeline of tasks in a project. Each horizontal bar in the chart reflects the start and end date of a specific activity, allowing stakeholders to track project progress and dependencies visually.

The Gantt chart covered:

- Training and Requirement Phase
- Test Plan Design and Tool Setup
- Script Development
- Testing and Debugging
- Reporting and Documentation

Fig 3.4 – Gantt Chart Representation of IMS AXES Project

Chapter 4: Project And System Requirement Study

4.1 User Characteristics

The IMS AXES project is developed for a range of user types involved in the software lifecycle. Each type of user interacts with the system from a different perspective and for different goals:

- *Hardware Developers:*

  These users are responsible for validating the functional requirements of the hardware integrations (if any) and ensuring that system compatibility is maintained.

- *Production Team:*

  This team uses the automation framework for end-to-end product testing before deployment. Their main objective is to ensure that the product meets all defined quality standards.

- *Sales and Marketing Team:*

  They use the tested and stable application to conduct demonstrations and deliver reliable showcases to prospective clients. A well-tested app adds confidence in client presentations.

- *Clients and End Users:*

  Clients benefit from a fully customized and tested version of the product. They rely on a bug-free and efficient application as part of their business operations. Their feedback helps in iterative testing and improvements.

4.2 Hardware And Software Requirements

To run the AXES Automation Framework smoothly, a predefined set of hardware and software resources are required. These requirements ensure optimal performance and support across environments.

Software Requirements:

- **Operating System:** Windows 10/11 or compatible Linux distribution
- **Development IDE:** Eclipse IDE (or any Java-compatible IDE)
- **Java (JDK):** Version 17 or higher
- **UFT:** Window application automation tool
- **TestNG:** For structuring test suites and reporting test execution
- **Maven:** For dependency management and build automation
- **Extent Reports:** For enhanced HTML-based test reporting
- **Browser Drivers:** ChromeDriver, GeckoDriver, etc.

Hardware Requirements:

- **Processor:** Minimum 3.5 GHz (Quad Core or higher preferred)
- **RAM:** Minimum 8 GB (16 GB recommended for smoother execution of parallel test suites)
- **Storage:** At least 512 GB SSD for efficient read/write performance
- **Display:** Full HD (1920x1080) recommended for UI validation tasks
- **Network:** Reliable internet for downloading dependencies and executing cloud-based tests (if applicable)

4.3 Assumptions and Dependencies for Projects

Assumptions:

- No other heavy applications are running on the system during test execution.
- The system running tests has administrative privileges.
- Auto-lock, screen saver, or sleep mode is disabled during script execution.
- Browser drivers are pre-installed and properly linked in the system PATH.
- The system has uninterrupted internet access for Maven and WebDriver updates.

Dependencies:

- *Reusable Function Libraries:*
  - Core logic components used throughout multiple test modules
  - Handlers for dynamic dropdowns, calendars, modal dialogues, etc.
- *Main Test Scripts:*
  - Calls to reusable functions and modules for test scenarios
  - Structured around the Page Object Model (POM)
- *Test Data Files:*
  - Excel/CSV-based parameter files used for data-driven testing
  - Environment-specific data (staging, dev, production)
- *Configuration Files:*
  - config.properties for dynamic environment settings
  - Logging and reporting setup
- *Report Viewers and Log Analysers:*
  - Tools like Extent Reports or custom HTML reports to analyse results
- *Google Looker Studio:*
  - It's a free tool that turns your data into informative, easy-to-read, and shareable dashboards and reports. It allows users to connect to hundreds of data sources, visualize data insights, and make data-driven decisions.

This combination of hardware/software setup and well-defined user expectations ensures that the IMS AXES framework can be executed with high reliability and scalability across environments.

Chapter 5: System/Resource Analysis

5.1 Study of Current System Resources

At the time of developing the IMSAxes Automation project, the internal tool/utility intended to automate the execution of performance and functional scripts was not fully established. Additionally, the main web application under test was itself in an evolving development phase. This led to some constraints while framing reusable test modules and scenarios, as several UI components and workflows were still changing.

5.2 Current System Problem and Weakness

- **Instability in Unexpected Scenarios:** The existing framework exhibited significant vulnerability to abrupt environmental interruptions, such as power transients, unexpected network disconnections, or forced system shutdowns. Due to the lack of dynamic
  state-saving mechanisms and automated recovery protocols, any terminal failure during execution resulted in aborted operations, potential data corruption in test logs, and the necessity for complete manual restarts, thereby reducing overall system reliability.

- **Incomplete Automation Readiness:** The automation suite demonstrated a lack of resilience against the rapid deployment cycles of the base Windows application. Because the scripts relied heavily on static locators, any unannounced modifications to the underlying UI architecture inevitably led to broken test flows ("flaky tests"). This constant need for manual script refactoring and recalibration created continuous maintenance overhead, severely hindering the timeline for achieving complete "automation readiness."

- **Compatibility Limitations:** The integration architecture struggled with seamless cross-environment operability when managing WebDriver instances. Technical bottlenecks were frequently observed during complex session handling-particularly when context-switching between different test domains or attempting to maintain persistent sessions across varied browser engines.
  This incompatibility often resulted in synchronization artifacts, stale element references, and premature session timeouts.

5.3 Requirements of New System

5.3.1 Functional Requirements

- **Module 1: Automated Fault Tolerance and Recovery (Robust Recovery Handling)**
  - Built-in exception handling to detect Windows application crashes and driver connection drops.
  - Ability to automatically relaunch the application .exe and continue execution after an unexpected closure.

- Significant reduction in false test failures caused by OS-level interruptions like Windows pop-ups or focus losses.

- **Module 2: Orchestrated Execution Environment (Enhanced Execution Utility)**
  - Completely independent script execution on dedicated Windows virtual machines without manual triggers.
  - Accurate statistics on total Windows tests run, passed, failed, or skipped due to hard application hangs.
  - Support for batch execution of grouped test cases against specific Windows application versions.
  - Search filters to rapidly locate specific scripts or historical Windows desktop execution logs.

- **Module 3:Intelligent Telemetry and Logging Mechanism**
  - Detailed and structured logs capturing simulated mouse/keyboard inputs and Windows Control ID interactions.
  - Failure cause indication with dumps of the Windows UI Automation tree for trace-level debugging.
  - Execution telemetry accessible across teams centrally, avoiding the need for Remote Desktop (RDP) to test machines.

- **Module 4:Desktop Data Visualization and Result Reporting**
  - Friendly UI dashboards for visualizing desktop test execution trends and pass/fail statistics.
  - Accurate reporting that automatically attaches full desktop or active window screenshots at the exact moment of failure.
  - Enable developers to quickly identify if failures stem from UI rendering issues, background process crashes, or backend data errors.

5.3.2 Non-Functional Requirements

- **Reliability:** Stable automation execution under continuous load, maintaining a persistent connection to the Windows desktop session without unexpected driver disconnects or framework crashes.
- **Performance:** Low-latency interaction with Windows UI elements and strictly optimized memory usage to prevent OutOfMemory (OOM) exceptions on the host machine during

prolonged test batches.

- **Scalability:**Easily extensible to test across multiple Windows operating systems (e.g., Windows 10, Windows 11) and adaptable to new underlying desktop technologies (such as WPF, WinForms, or UWP).

- **Usability:** Windows object repositories and automation scripts should be highly modular and reusable, with centralized configuration files that make it easy for QA teams to deploy and run tests across different local environments.

5.4 Feasibility Study

5.4.1 Contribution to Organisation Goals

Yes. IMSAxes Automation directly supports the organization's objective of minimizing manual QA effort while ensuring high reliability in our Windows desktop applications. It significantly accelerates regression testing cycles, reduces human error, and directly contributes to faster, more confident product release timelines.

5.4.2 Technical and Economic Feasibility

Yes. The system can be successfully engineered using reliable, currently available technologies such as Java, WinAppDriver (or UIAutomation), Maven, and Jenkins within acceptable timeline constraints. The overall economic implication is highly
favorable, as it utilizes predominantly open-source tooling, with primary costs relegated strictly to execution infrastructure (e.g., Windows Virtual Machines) and initial development hours.

5.4.3 Integration Feasibility

Yes. The proposed Windows automation framework is inherently modular. It can be seamlessly integrated into existing CI/CD pipelines via Windows runner nodes, allowing tests to be triggered automatically upon new builds. Furthermore, its architecture can be easily adapted to support other internal Windows desktop projects or domains in the future.

5.5 New System Features

- Fully resilient Windows desktop automation execution, capable of recovering from unhandled application crashes and OS-level interruptions.
- Centralized, timestamped telemetry logging that captures every simulated Windows UI interaction (mouse/keyboard events) for all test scenarios.
- Support for unattended, batch-based execution of Windows test suites across

distributed virtual machines.

- Interactive GUI-based dashboards providing pass/fail trend charts, granular filters, and active-window screenshots for highly traceable defect analysis.

5.6 Function of System

- Admin Role:
    - Assign specific Windows automation modules (e.g., script creation or maintenance) to QA Engineers based on the project requirements.
    - Review aggregate quality metrics and approve final test execution reports for release readiness.
    - Manage employee profiles and monitor assigned task statuses.
- Employee Role:
    - View, configure, and execute assigned Windows desktop test scripts locally or on remote nodes.
    - Track the progress of executed test suites and utilize centralized logs and screenshots to debug individual script failures.
    - Update the operational status of test cases (e.g., Passed, Failed, Needs Refactoring) and directly log defect root causes based on system feedback.Modify personal profiles and progress logs.

5.7 ACTIVITY DIAGRAM

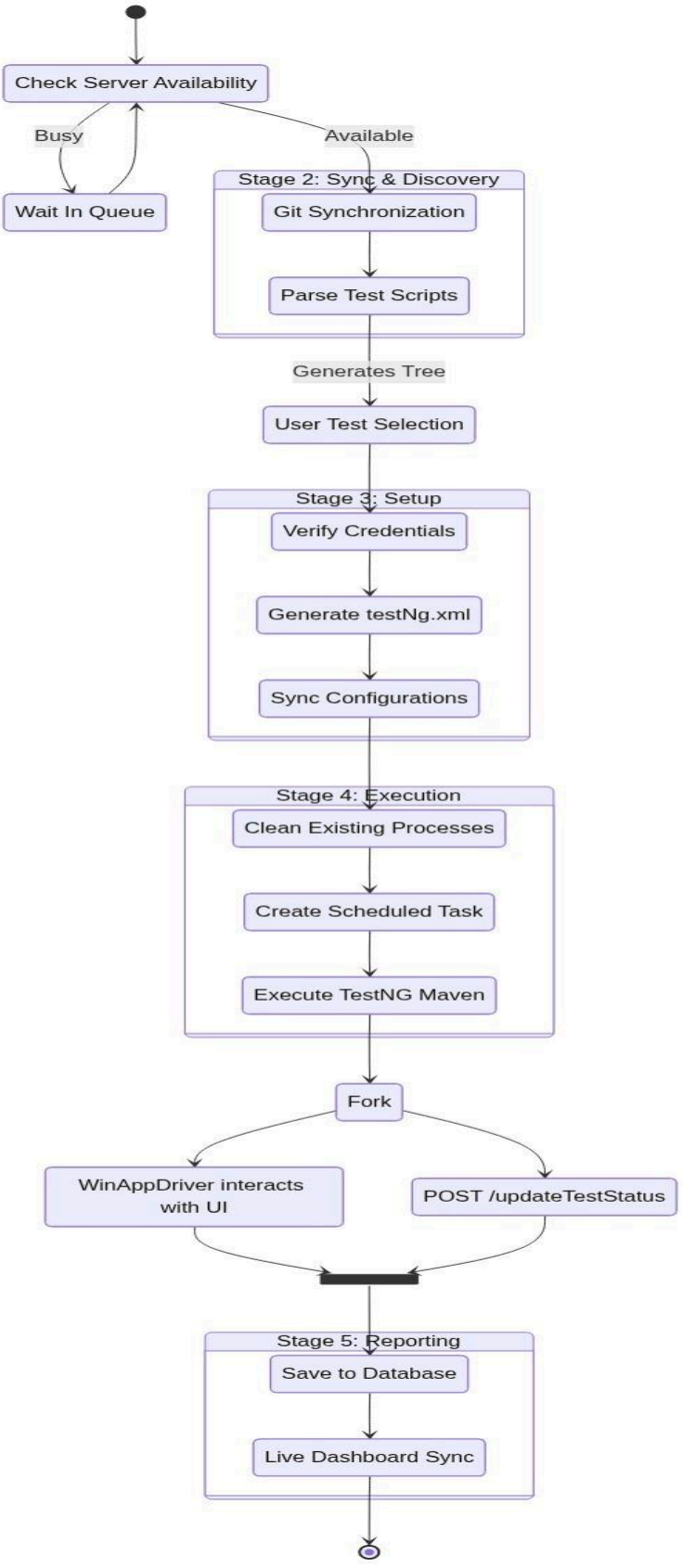


Fig. 5.1 – Activity Diagram for Automation Testing

5.8 USE CASE DIAGRAM

Depicts how different roles interact with the IMSAxes Automation Portal.

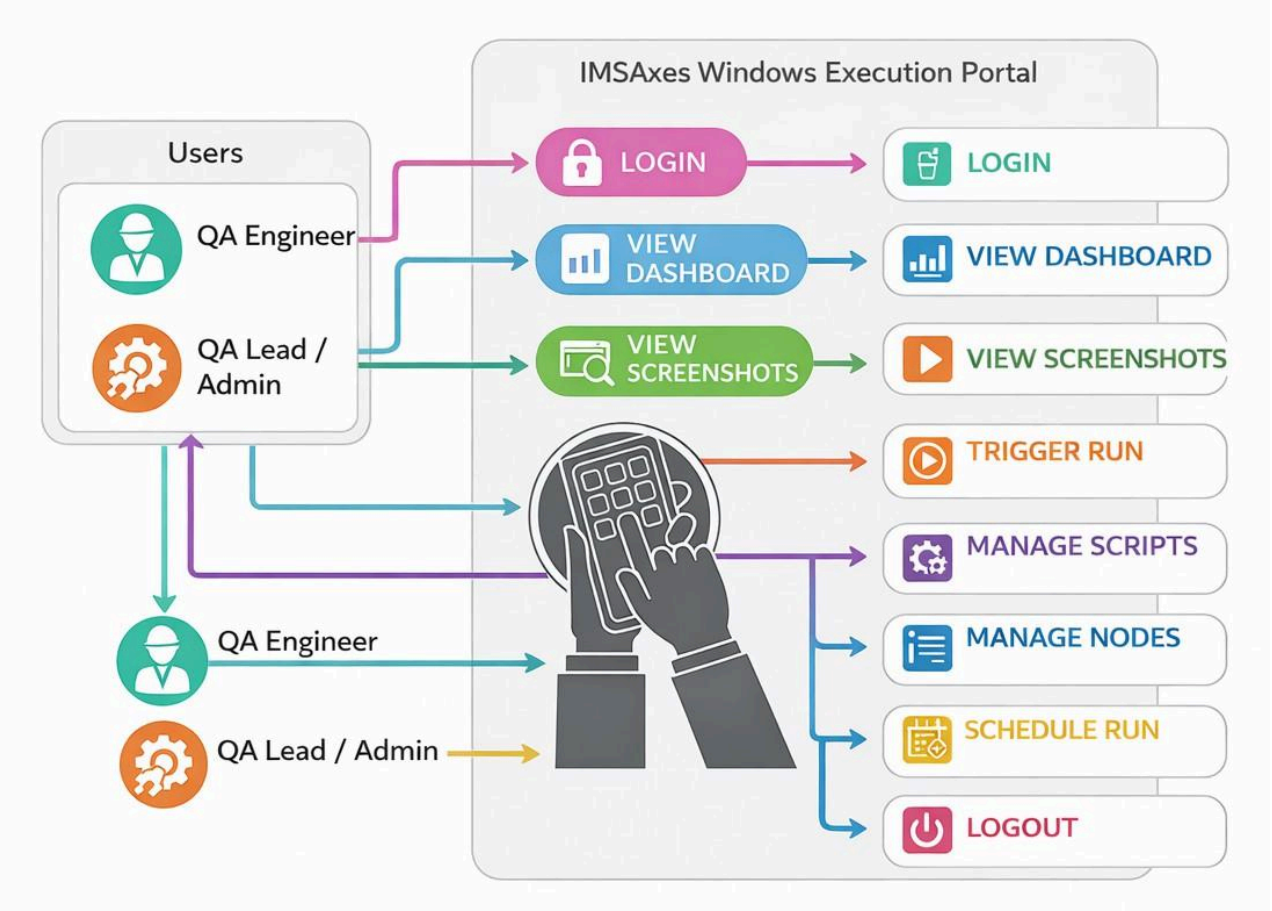


Fig. 5.2 – Use Case Diagram for IMAAxes Automation Portal

5.9 FRAMEWORK DIAGRAM

Outlines the layered architecture of the IMSAxes Automation testing framework.

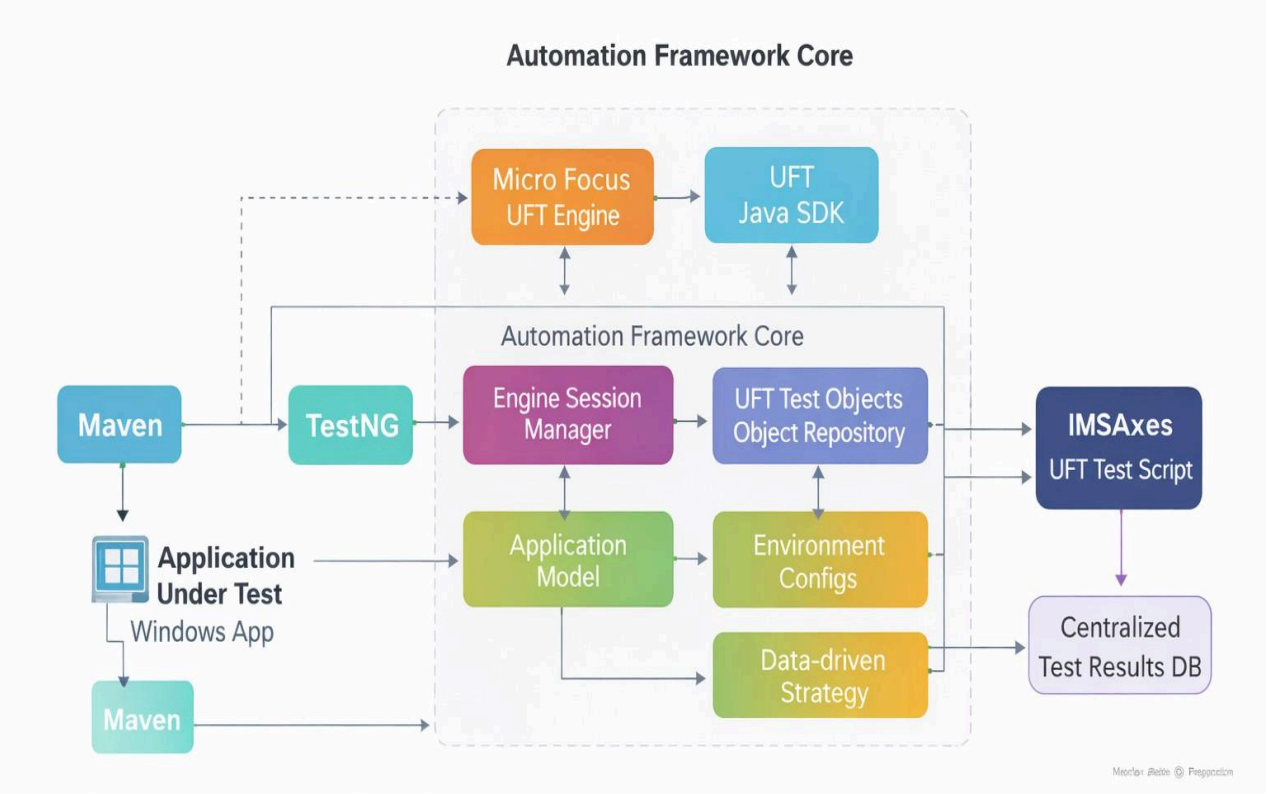


Fig. 5.3 – Framework for Automation Testing

5.10 SEQUENCE DIAGRAM

Describes how the automation testing flow operates step-by-step from test trigger to execution and result capture.

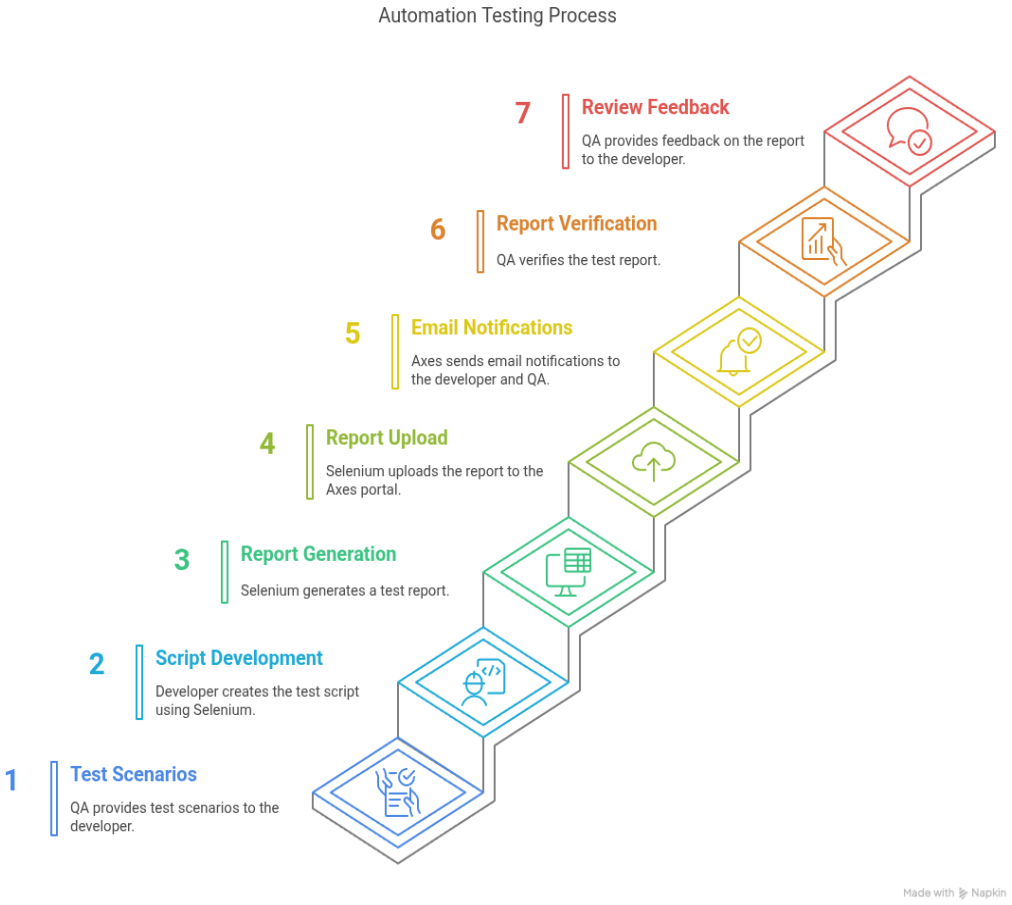


Fig. 5.4 – Sequence Diagram for Automation Testi

Chapter 6: IMSAxes Design

6.1 Login Page Design

The IMSAxes Automation Portal features a secure, React-based web interface designed for seamless interaction with the underlying remote Windows execution engines. The central login component and navigation routing are strictly optimized for QA Engineers and Automation Leads, enabling them to securely manage UFT test scripts, trigger remote execution batches, and monitor real-time telemetry without exposing the remote servers to unauthorized access.

The following figures represent the critical design validations and secure user interface interactions parameterized within the IMSAxes portal:

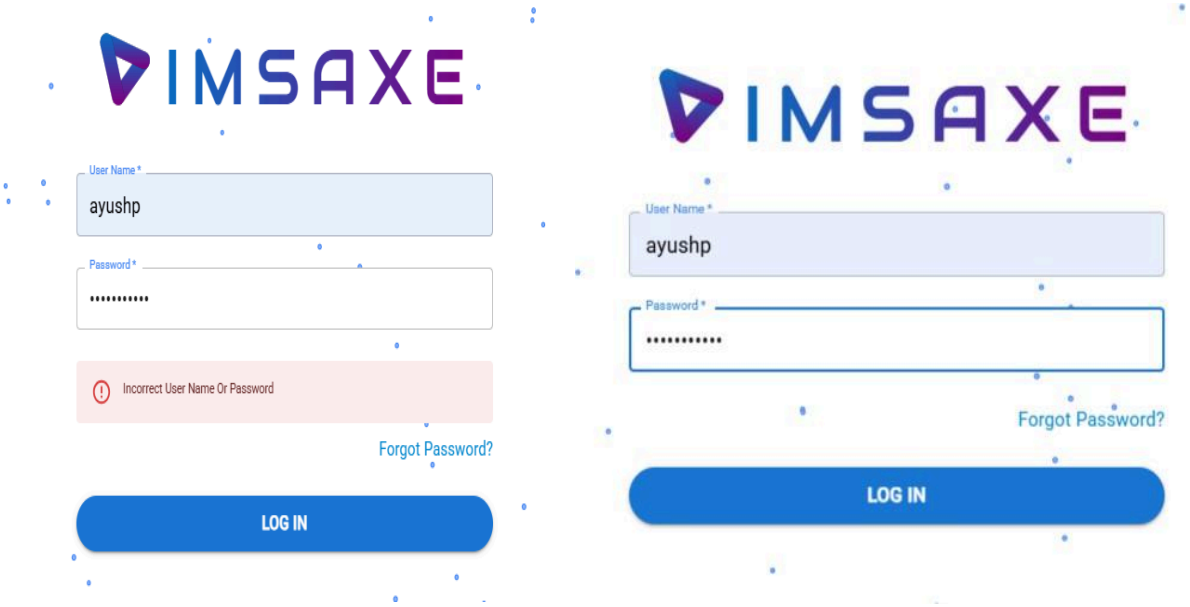


Fig 6.1 – Login Attempts

- Enter a username and an incorrect password.
- Click the "Login" button.
- Verify whether access is granted or an error message appears.

Fig 6.2 – Verify Forgot Password

Fig 6.3 – Verify Incorrect Credentials During Password Recovery

- Click on the "Forgot Password" link.
- Enter an incorrect or non-existent username.
- Validate whether the system shows an appropriate authentication error.

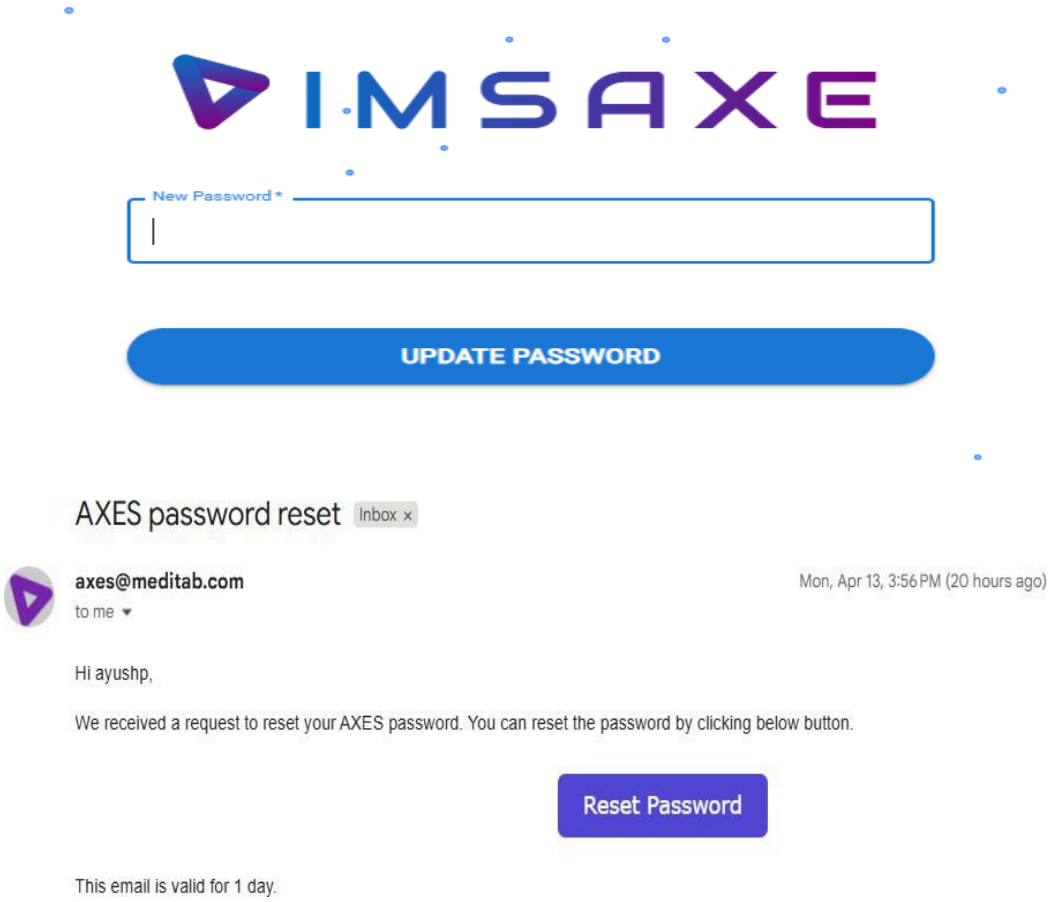


Fig 6.4 – Update Password

- Enter the correct username.
- Click on the "Reset Password" button.
- Confirm that a password reset pop-up is triggered as expected.

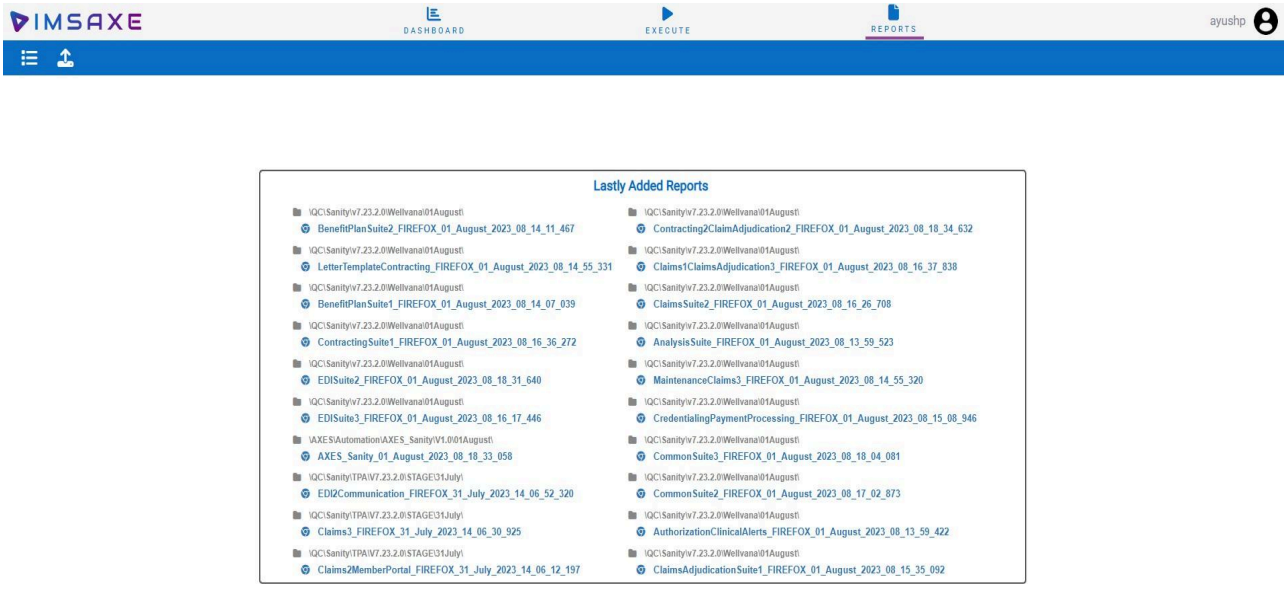


Fig 6.5 – Home Page on Successful Login

- Enter the correct username and password.
- Click "Login."
- Verify that the user is redirected to the dashboard or home page with available modules.

These interface checkpoints are important in ensuring that the user experience remains consistent and that all edge cases related to authentication are validated through automation scripts.

Screenshots and UI verification are typically performed during sanity and regression testing of the IMSAxes portal.

6.2 Interface Design

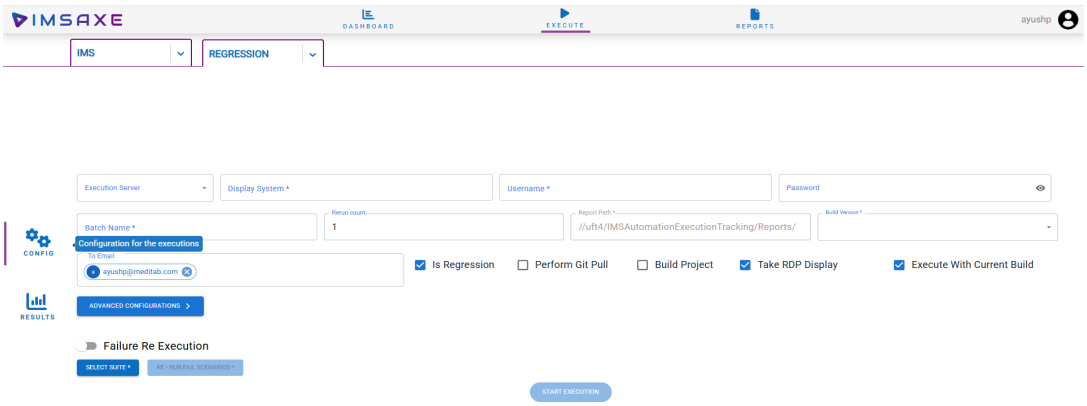


Fig 6.6 – Configuration Page

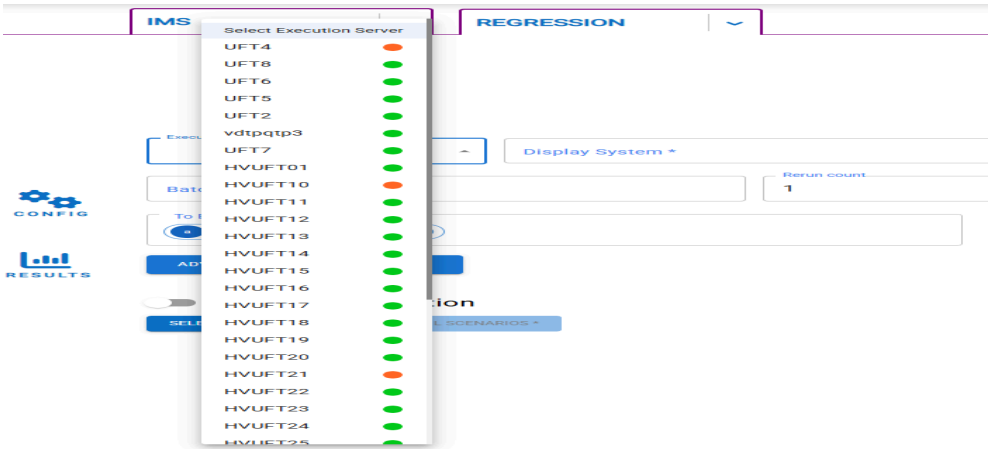


Fig 6.7 – Server Selection

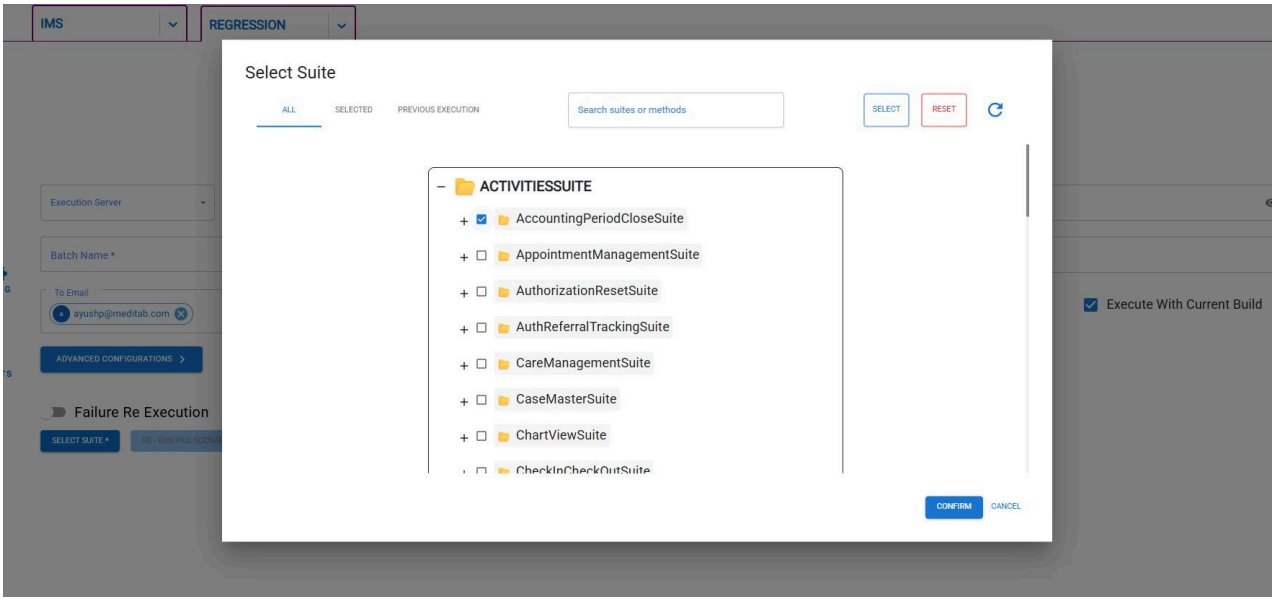


Fig 6.8 – Suite Selection

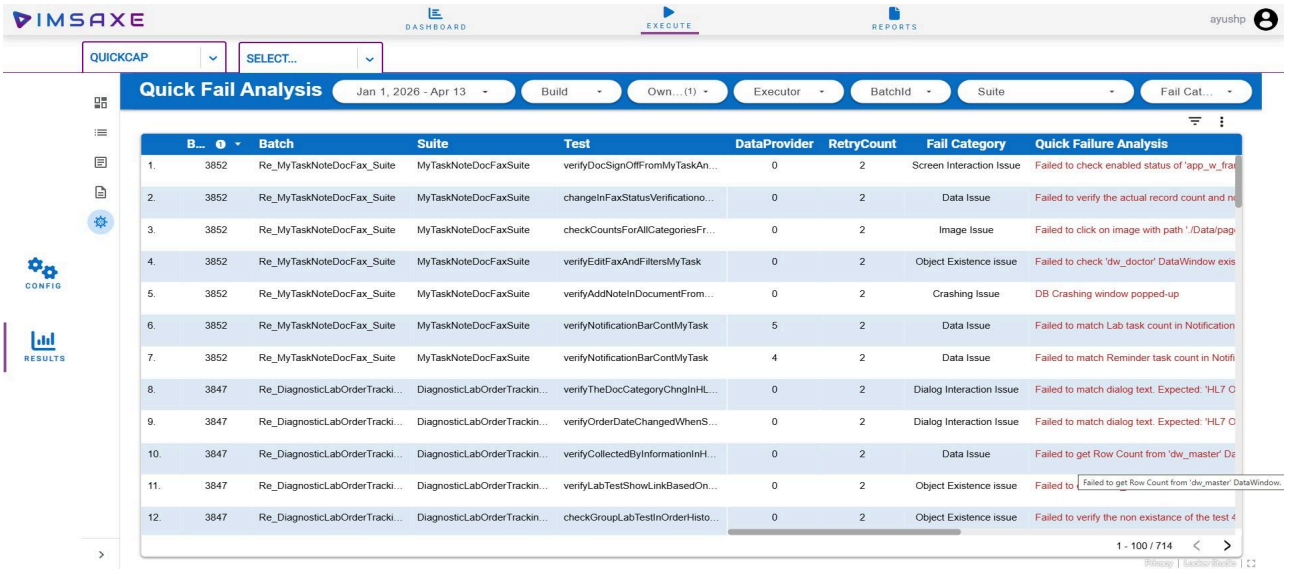


Fig 6.9 – Report Page

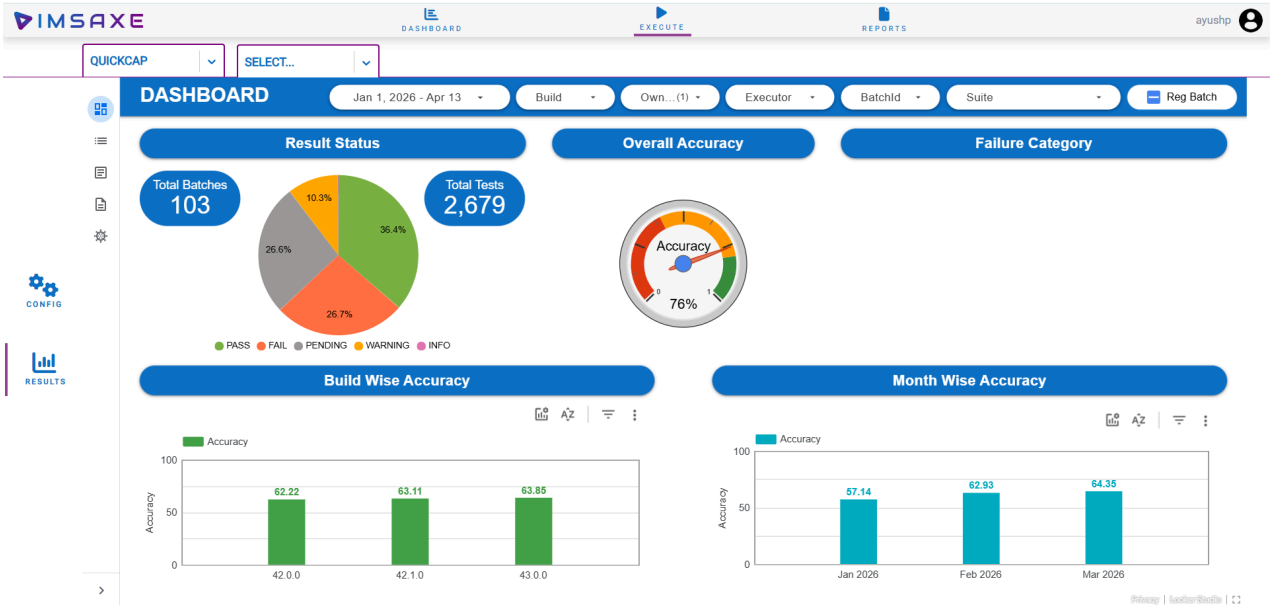


Fig 6.10 – Analysis Page

Chapter 7: Implementation Planning

7.1 Implementation Environment

For efficient and scalable Windows automation testing, a well-configured implementation architecture is crucial. The IMSAxes Automation framework relies on a decoupled, dynamic execution portal and a structured network of remote runner nodes to ensure smooth "lights-out" automation out-of-hours.

Environment Setup Requirements:

- *Automation Utility Interface:*
  A centralized, React-based web dashboard (the IMSAxes Portal) is required to orchestrate execution. It enables QA Engineers to trigger multiple UFT execution batches across remote virtual machines without requiring manual Remote Desktop Protocol (RDP) access, drastically improving overall testing productivity.

- *Client Scenario Replication:*
  The remote Windows machines (imsautomate02, etc.) must maintain cloned databases tailored to mimic the exact database size, schema, and registry configurations of a live clinical medical practice. This accurately replicates real-world usage of the IMS application and ensures realistic automation outcomes.

- *System Configuration:*
  - **High-performance hardware** is strictly required, as running the native Windows IMS.exe executable concurrently with the background Micro Focus UFT Engine consumes substantial system memory.
  - Intel Xeon E5 or equivalent, 16 GB RAM, 100 GB SSD storage.
  - The primary remote test machines must operate in isolated sessions natively during script execution to prevent OS-level focus loss or unhandled mouse-cursor interruptions.

- *Parallel Setup:*
  - To maintain execution continuity, dedicated Virtual Machines are segregated into strictly "Execution" and "Development" nodes. An additional system must be allocated solely for automation engineers to maintain and debug new Java/UFT scripts without interrupting live nightly regression suites.

This setup facilitates efficient test execution, reduces downtime, and improves resource utilisation in the testing lifecycle.

7.2 Program / Modules Specifications

As part of the internal IMSAxes project architecture, code was developed across both the web orchestration portal and the corresponding backend automation scripts. The core engineered modules include:

- **Secure Authentication Module:** Implementing JWT-based secure routing for QA personnel.
- **Environment & Test Discovery Module:** Node.js logic that parses remote Java TestNG repositories via regex to dynamically generate test execution trees.
- **Windows Process Manager:** PowerShell scripting integration that actively hunts and executes Taskkill on stale Windows processes (e.g., hanging ims.exe or uft.exe instances).
- **Real-time HTTP Telemetry:** Socket.IO and REST endpoints that consume live trace statements from the remote UFT execution and pipe them directly into the MySQL tracking database.
- **Dynamic Visual Dashboard:** Providing instant analysis on Pass/Fail metrics and providing direct hyperlinks to execution error screenshots.

Each architectural module was developed with strict micro-service principles and modular scalability in mind, enabling cross-client adaptability regardless of the specific test suites being executed.

7.3 Coding Standards

To maintain strict readability, reusability, and consistency across both the Node.js/React portal and the Java testing codebase, the following coding standards were strictly enforced:

- Naming Conventions:
  - Implemented universal CamelCase styling for variables and automation methods.

```
(checkUftEngineStatus( ) ,triggerMavenJob())
```

- Function Size:
  - Methods were kept intentionally short and focused on a single logical responsibility (Single Responsibility Principle) to enhance testability and limit debugging scope.
- Code Simplicity:
  - Engineers adhered strictly to Agile principles, developing only what was architecturally necessary for current sprint requirements and avoiding convoluted premature optimization.

- Reusability & encapsulation:
    - Frequently utilized UI operations were abstracted away from the business layer. In the context of UFT, Windows API interactions and static waits were encapsulated into a shared Application Model and global utility functions to prevent script duplication.

These unified standards effectively guaranteed a clean, extensible automation platform capable of scaling seamlessly alongside future testing requirements.

Chapter 8: Testing

8.1 Testing Plan

Testing is a critical phase in the automation project lifecycle, ensuring that the IMSAxes Automation framework consistently delivers reliable results across multiple QA scenarios, Windows environments, and execution nodes. The comprehensive testing plan for the IMSAxes project encompasses both validating the accuracy of the automation test results and ensuring the performance scalability of the underlying Node.js orchestration backend.

Automation Testing Plan

- **Automation Tools Required:**
  - Micro Focus UFT Engine (Unified Functional Testing) & UFT Java SDK.
  - TestNG Framework and Maven for dependency and lifecycle management.
  - PowerShell for OS-level execution orchestration.
  - React & Express (Node.js) for telemetry capture and centralized dashboard reporting.
  - Eclipse / IntelliJ IDE for Java script development.

- **Framework Characteristics:**
  - Modular and Reusable: Scripts are highly decoupled from business logic.
  - Application Model Architecture: Officially implements UFT's Application Model (similar to the Page Object Model), utilizing robust Object Repositories instead of brittle dynamic locators.
  - Data-Driven and Batch Support: Supports massive parallel execution clusters and dynamic data injection from external SQL/Excel environments.
  - Real-Time Telemetry: Bypasses traditional static HTML reports by streaming execution data over HTTP callbacks directly into an interactive React dashboard.

- **Scope Definition:**
  - **In-Scope:**
    - Functional testing of the target native IMS Windows application workflows (Login, Patient Registration, etc.).
    - Validation of the standalone IMSAxes Web Portal (Task management,

trigger buttons, reporting modules).

- Cross-OS execution testing (testing the framework against Windows 10 vs. Windows Server deployments).
- Automated nightly build Regression and Sanity checks.

- **Out-of-Scope:**
  - Non-UI backend database structural validations.
  - Deep security penetration testing of the corporate network.

- **Test Suite Preparation:**
  - Structured, hierarchical test suites (testNg.xml) are generated dynamically by the backend.
  - Data sets are parameterized securely using centralized configuration files deployed via PowerShell.
  - Test suites are rigidly organized into predefined clusters for scheduled unattended nightly runs vs. specific on-demand interactive runs.

- **Scripting and Execution Schedules:**
  - Weekly UFT script reviews and Object Repository updates based on newly released IMS.exe application features.
  - Daily remote execution for continuous regression and sanity validation.
- **Automation Testing Deliverables:**
  - Robust UFT/Java automation test scripts.
  - Modular Windows UI utility functions and Object Repositories.
  - Live centralized execution telemetry traces stored in MySQL.
  - A fully interactive React summary dashboard for upper-management stakeholder visibility.

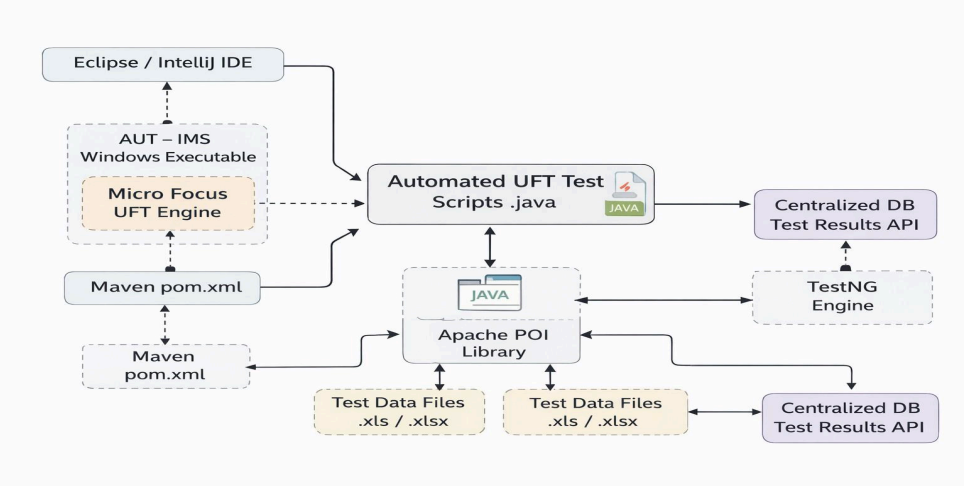


Fig. 8.1 – Automation Testing Flow Chart

Performance Testing Plan

The performance testing strategy was designed to ensure that the newly built IMSAxes web portal and its backend Node.js architecture could scale efficiently without dropping crucial telemetry data during massive parallel execution batches. Key performance metrics include API response times, WebSocket throughput handling, and Database read/write latency.

- **Load Testing*:***
  - Simulate high-volume HTTP callback traffic (simulating over 1,000 parallel test executions posting results simultaneously) to verify that the Express backend processes the payload and returns a status 200 OK in under 2 seconds.
- **Stress Testing:**
  - Gradually increase WebSocket connections (simulating an excessive number of attached remote runner nodes) to determine the threshold before the Node server crashes or begins dropping live broadcast events.
- **Network Simulation:**
  - Test the React frontend and VNC proxy responsiveness in artificially throttled, low-bandwidth intranet scenarios.
- **Database Load Testing:**
  - Execute massive continuous asynchronous INSERT operations (e.g., simulating 500 test cases failing and dumping massive stack traces simultaneously) to monitor latency limits in the MySQL database.
- **System Resource Monitoring:**
  - Measurably track Windows CPU spikes and memory consumption footprints on the remote testing VMs when running ims.exe concurrently alongside the uft.exe engine.
- **Scalability and Stability:**
  - Verify the response time and overall stability of the dashboard under low, moderate, and absolute peak testing load conditions (e.g., during end-of-month regression sweeps

8.2 TESTING STRATEGY

For Automation Testing

The strategic approach focuses intensely on identifying high-ROI (Return on Investment) test cases that are the strongest candidates for desktop automation:

- Tests cases that must be identically repeated across multiple weekly internal application builds.
- Workflows that are mathematically complex and highly susceptible to human fat-finger errors during manual data entry.
- Clinical scenarios requiring significantly large dynamic datasets (e.g., mass billing simulations).
- Core application workflows (Smoke tests) representing high-risk business functionalities.
- Complex desktop verification cases that are simply infeasible for manual QA to complete within the 48-hour release window.
- Time-consuming manual procedures that tie up critical QA resources unnecessarily.

This overarching strategy guarantees that IMS releases are vetted consistently, rapidly, and natively at an enterprise scale with every subsequent patch cycle.

Chapter 9: Conclusion

9.1 Problems Encountered and Possible Solutions

For Performance and Backend Scalability:

Encountered Problem: A critical performance bottleneck was observed during the initial development of the telemetry logging system. When massive parallel test suites executed simultaneously, the high volume of incoming HTTP Post payload broadcasts caused the Node.js event loop to block, leading to delayed dashboard updates and database timeout errors.

Implemented Solution: Following a deep architectural investigation, the backend payload processing was restructured asynchronously. Real-time updates were offloaded exclusively to lightweight Socket.IO channels, and complex MySQL insert queries were refactored into optimized, bulk-batched INSERT operations to drastically reduce connection overhead.

For Automation Testing Connectivity:

Encountered Problem: Early iterations of the Windows runtime execution suffered from "ghost executions," where stale uft.exe or ims.exe background processes from previous failed runs prevented new automation batches from launching correctly, requiring tedious manual intervention via Remote Desktop.

Implemented Solution: A dynamic PowerShell process-hunting mechanism (start_execution.ps1) was engineered to systematically aggressively execute a Taskkill against any orphaned target processes prior to triggering the Maven jobs, ensuring a highly sterile execution environment.

For Script Architectural Maintenance:

Encountered Problem: Legacy automation scripts contained significant redundancy relying heavily on primitive WinApp locator strategies, causing severe maintenance overhead whenever the target Windows application updated its UI framework.

Implemented Solution: The codebase was heavily abstracted to strictly utilize the UFT Application Model (Object Repository). This fundamentally detached business logic from

object identification, ensuring that a single UI update only required a single tweak within the repository rather than hundreds of script edits.

9.2 Summary of Internship Work

The performance testing component taught us the importance of scalable application design and proactive load simulation. Testing under various user loads and response times helped us verify system integrity under stress conditions.

- **Saved Critical Testing Time:** Achieved by enabling unattended, parallel batch executions across remote virtual machines.
- **Reduced Manual Errors:** Eliminated the risks associated with manual environment configuration and human data-logging.
- **Increased Team Productivity:** Provided QA engineers with a centralized, "single pane of glass" dashboard to effortlessly monitor global test results.
- **Ensured Consistent Testing:** Locked automation executions into a strict, version-controlled repository to guarantee precision across all future software builds.

The project demonstrated that both automation and performance testing are crucial for modern, client-facing applications.

Chapter 10: Limitations and Future Enhancements

10.1 Limitations

- Cannot Replace Exploratory Intelligence: Automation scripts execute strictly defined paths. They cannot replace the human intuition required for exploratory testing, nor can they accurately evaluate subjective qualities like front-end user experience (UX) and
  aesthetic design.
- Susceptibility to Windows UI Volatility: Despite utilizing robust UFT Object Repositories, significant architectural changes to the underlying Windows .exe forms often result in stale object references. This requires diligent and regular maintenance of the automation codebase.
- 100% Automation is Infeasible: Designing scripts for highly complex, edge-case medical scenarios often yields diminishing returns. Achieving 100% automation coverage is practically infeasible due to the sheer cost, time, and escalating architectural complexity required to maintain it.

10.2 Future Enhancements

- AI-Assisted Test Generation: Developing a machine-learning utility capable of automatically scaffolding boilerplate UFT automation scripts and Object Repository
  mappings directly from plain-text BDD (Behavior-Driven Development) test case inputs.
- Predictive Data Generation: Implementing dynamic, AI-based mock data generators that automatically seed the required SQL databases with randomized, anonymized medical
  profiles prior to script execution.
- Automated Performance CI/CD Integration: Expanding the current functional pipeline to integrate backend load-testing dynamically, automatically halting deployments if the Node.js API performance degrades beyond acceptable network metrics.
- Predictive Dashboard Analytics: Enhancing the current React dashboard with AI anomaly detection, allowing the UI to flag irregular execution times or recurring failure trends proactively, rather than simply displaying raw pass/fail statistics.

Chapter 11: Bibliography/ References

For reference, I used the following websites for help:

(https://admhelp.microfocus.com/leanft/)

(https://react.dev/)

(https://socket.io/docs/v4/)

(https://nodejs.org/en/docs/)

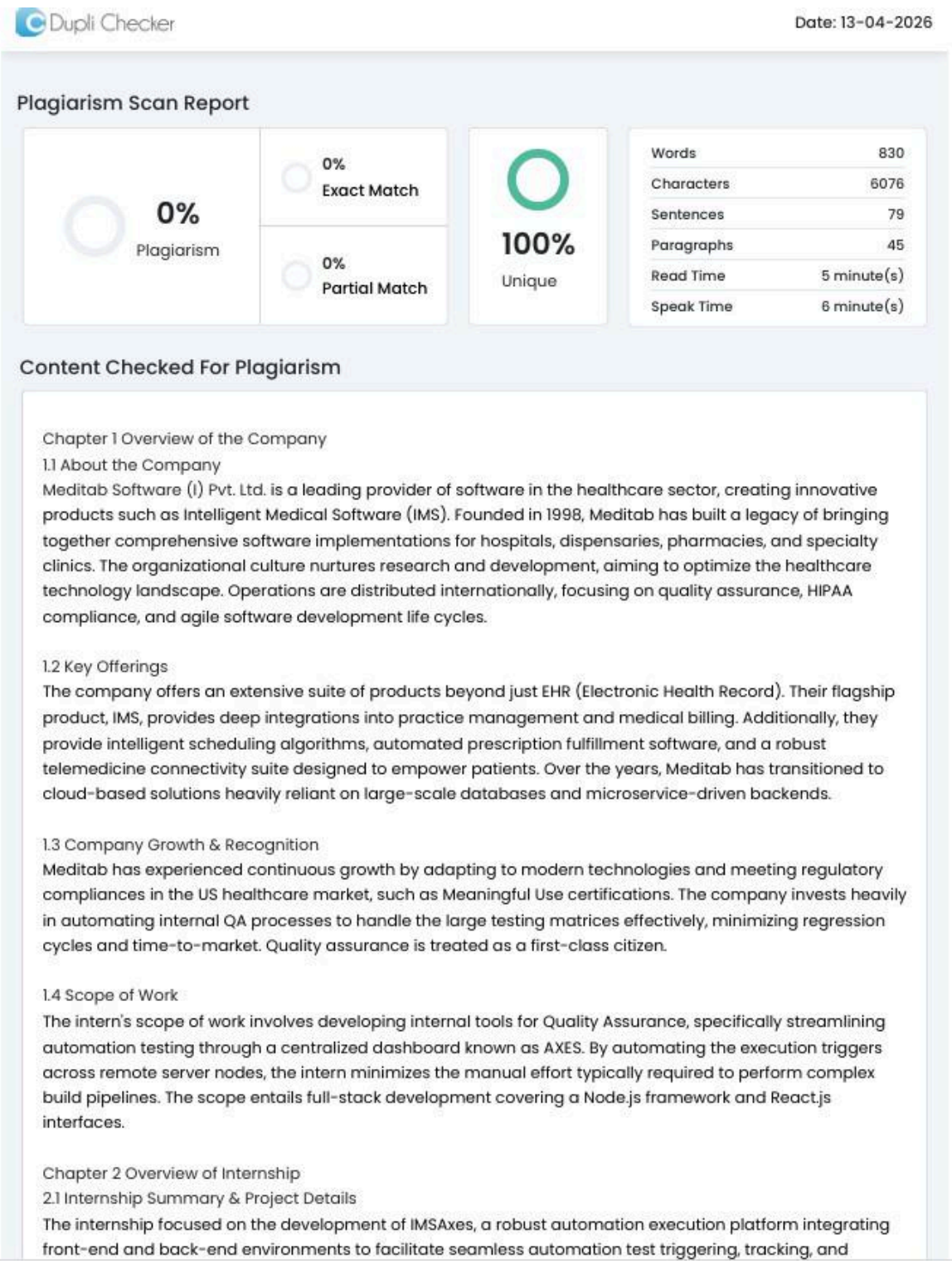
(https://testng.org/doc/)

(https://maven.apache.org/)

(https://mvnrepository.com/)

(https://www.tutorialspoint.com/software_testing_dictionary/performance_testing.html)

Chapter 12 – Appendix



Dupli Checker

Date: 13-04-2026

Plagiarism Scan Report

0% Plagiarism	0% Exact Match
	0% Partial Match
100% Unique	

Words	830
Characters	6076
Sentences	79
Paragraphs	45
Read Time	5 minute(s)
Speak Time	6 minute(s)

Content Checked For Plagiarism

Chapter 1 Overview of the Company

1.1 About the Company

Meditab Software (I) Pvt. Ltd. is a leading provider of software in the healthcare sector, creating innovative products such as Intelligent Medical Software (IMS). Founded in 1998, Meditab has built a legacy of bringing together comprehensive software implementations for hospitals, dispensaries, pharmacies, and specialty clinics. The organizational culture nurtures research and development, aiming to optimize the healthcare technology landscape. Operations are distributed internationally, focusing on quality assurance, HIPAA compliance, and agile software development life cycles.

1.2 Key Offerings

The company offers an extensive suite of products beyond just EHR (Electronic Health Record). Their flagship product, IMS, provides deep integrations into practice management and medical billing. Additionally, they provide intelligent scheduling algorithms, automated prescription fulfillment software, and a robust telemedicine connectivity suite designed to empower patients. Over the years, Meditab has transitioned to cloud-based solutions heavily reliant on large-scale databases and microservice-driven backends.

1.3 Company Growth & Recognition

Meditab has experienced continuous growth by adapting to modern technologies and meeting regulatory compliances in the US healthcare market, such as Meaningful Use certifications. The company invests heavily in automating internal QA processes to handle the large testing matrices effectively, minimizing regression cycles and time-to-market. Quality assurance is treated as a first-class citizen.

1.4 Scope of Work

The intern's scope of work involves developing internal tools for Quality Assurance, specifically streamlining automation testing through a centralized dashboard known as AXES. By automating the execution triggers across remote server nodes, the intern minimizes the manual effort typically required to perform complex build pipelines. The scope entails full-stack development covering a Node.js framework and React.js interfaces.

Chapter 2 Overview of Internship

2.1 Internship Summary & Project Details

The internship focused on the development of IMSAxes, a robust automation execution platform integrating front-end and back-end environments to facilitate seamless automation test triggering, tracking, and